

Development: AdCraft — Structure-Aware Ad Copy Generation via Template Retrieval and Pairwise Preference Ranking

Kaiyang Teng
UIUC
kteng4@illinois.edu

Yipeng Xin
UIUC
yipengx2@illinois.edu

Hengrui Ren
UIUC
hengrui6@illinois.edu

Ziqin Shen
UIUC
ziqins2@illinois.edu

ABSTRACT

AI-powered copywriting tools have become standard in digital marketing, yet most operate as black boxes: they produce ad copy without exposing the rhetorical structure behind it. AdCraft addresses this by mining structural patterns from real Facebook ad copy, indexing 1,450 structural templates in a dense vector store, and ranking them using pairwise user preference data collected from volunteer annotators. The system provides two generation modes: a structured mode that retrieves a ranked template and generates copy with visible per-sentence pattern labels, and a direct mode that replicates unconstrained language model generation for comparison. Users can further edit individual segments without regenerating the entire copy. Design decisions are grounded in the Google PAIR Guidebook principles of explainability and user control, and in the interactive machine learning framework of Amershi et al. The deployed prototype is available at <https://adcraft.jnxy.net> and source code at <https://github.com/jnxy/AdCraft>.

INTRODUCTION

AI-powered copywriting tools are now widely used by marketers, small business owners, and content creators. A user provides a product description, and the tool returns finished ad copy in seconds. Despite their convenience, these tools share the same limitation: they just return the final text without any explanation. Users cannot see the structure behind the output, cannot edit individual parts of it, and do not learn anything about ad writing from using the tool.

This gap matters for three reasons. First, without visibility into structure, users cannot judge whether the chosen approach is appropriate for their audience. Second, changing one part of the copy typically requires regenerating the entire output. Third, because the writing logic stays hidden, the tool provides no learning value for users who want to improve their own copywriting skills.

The Google PAIR Guidebook identifies transparency and user control as foundational requirements for trustworthy AI systems [3]. Existing tools fall into two categories: unstructured generators such as Jasper AI and QuillBot, which apply no structural constraints and offer no sentence-level editing; and fixed-template tools such as Narrato, which use pre-defined formats with no connection to real ad data. AdCraft takes a different approach: structural templates are mined from real Facebook ads, dynamically retrieved to match the user’s product context, ranked by collected pairwise preference data, and the output is editable sentence by sentence with visible rhetorical labels. Specifically, AdCraft makes three concrete contributions:

1. **A rhetorical pattern taxonomy and annotation pipeline.** We represent each ad as an ordered sequence of ten rhetorical pattern codes, mined from real Facebook ad data using a large language model. The taxonomy is grounded in established advertising message strategy research [4].
2. **A template retrieval and ranking system.** We index 1,450 structural templates using dense semantic vectors and rank them

with a custom Bradley-Terry model [2] trained on volunteer pairwise preference data.

3. **A segment-level generation and editing interface.** Users see the pattern label for each sentence, can switch between generation modes, and can edit individual sections without regenerating the full copy.

The evaluation loop follows the interactive machine learning framework from Amershi et al. [1]: volunteer evaluators submit pairwise preferences, the model updates template scores, and future generation requests retrieve higher-ranked templates.

INTENDED USERS

AdCraft is designed for three groups of users:

Marketing practitioners and small business owners who write ad copy regularly but do not have a background in advertising theory. These users benefit from seeing effective structural patterns from real ads, which provides a concrete starting point rather than a blank page.

Content creators who want to understand why certain ad structures are more compelling than others. The visible pattern labels and side-by-side comparison of structured versus direct generation help users build intuition about advertising rhetoric.

Researchers and educators interested in studying rhetorical structure in persuasive text. The evaluation interface and template library provide a platform for collecting and analyzing pairwise structural preferences at scale.

SYSTEM OVERVIEW

AdCraft consists of three components: an offline data pipeline, a backend API server, and a web frontend. The system architecture is shown in Figure 1.

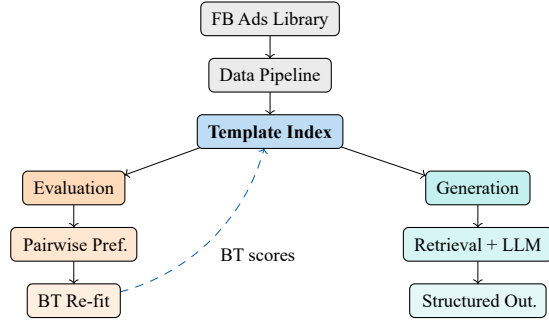


Figure 1: AdCraft system architecture.

The offline pipeline reads raw ad data and produces three artifacts: a set of annotated ads, a library of structural templates, and a set of pairwise evaluation tasks. The backend loads these artifacts at startup, indexes templates in ChromaDB, and exposes REST endpoints for evaluation and generation. The frontend is a single-page application with two views: an evaluation view for collecting pairwise preferences and a generation view for retrieving templates and producing structured copy.

The evaluation and generation paths share the same template index. Preference responses collected from the evaluation path drive periodic re-fitting of Bradley-Terry scores, which are written back into the index and influence what the generation path recommends.

DATA PIPELINE

The offline pipeline transforms raw Facebook ad data through five sequential stages: deduplication, annotation, clustering, template building, and evaluation task generation. The pipeline is implemented in Python and can be re-run incrementally; each stage checks whether its output artifact already exists and skips processing if nothing upstream has changed.

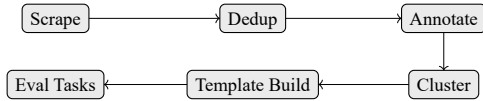


Figure 2: Data pipeline stages.

Data Collection

We applied to access the official Meta Content Library and API [5] but received no response. We therefore collected ads from the publicly accessible Facebook Ad Library using web scraping in April 2026. The scraper ran single-threaded with a 1.5-second delay between requests, keeping traffic well within normal browsing levels. Queries used 59 keyword phrases covering ecommerce, health and beauty, education, finance, travel, technology, food, and fitness verticals. Results were filtered to currently active ads served in the United States and sorted by total impressions in descending order. After the two-pass deduplication step described below, the corpus contains over 2,500 ads distributed approximately evenly across five major product categories (technology, health, finance, beauty, and consumer products), with roughly 400–500 ads per category. Each record includes the ad body text, product category, and advertiser identity.

Deduplication

Deduplication runs in two passes. The first pass removes exact duplicates by matching on body text. The second pass uses embedding-based approximate deduplication: body texts are embedded with *text-embedding-3-small* and any ad whose cosine similarity to an already-retained ad exceeds 0.97 is dropped.

LLM Annotation

We use a large language model to annotate each ad with an ordered sequence of rhetorical pattern codes. The taxonomy contains ten labels, grounded in prior advertising message strategy research [4]:

- AH** Attention Hook: opening line that grabs attention
- PP** Pain Point: audience’s concrete problem or friction
- AG** Agitation: amplification of the cost of inaction
- FB** Feature-Benefit: specific feature tied to user benefit
- SP** Social Proof: adoption figures, ratings, testimonials
- BA** Before-After: contrast of current vs. improved state
- AU** Authority: expert backing or credentials
- UR** Urgency: reason to act now
- OF** Offer: explicit value proposition
- CTA** Call to Action: direct action instruction

The annotator calls the language model concurrently with up to 20 parallel requests, processes ads in batches of ten, and supports checkpoint resume so that interrupted runs do not reprocess completed ads.

Semantic Clustering

We embed the inferred product description of each ad and cluster the embeddings to produce 117 semantic clusters. Ads within a cluster share similar product domain context, making within-cluster structural comparisons more meaningful as a preference signal.

Template Building

Ads are grouped by identical pattern sequence to form structural templates. Each template records the sequence, a human-readable name derived from the sequence, a length bucket, a frequency score indicating how often that sequence appeared in the corpus, and representative example text. The current library contains 1,450 distinct structural templates.

Evaluation Task Generation

The task builder generates pairwise comparison tasks by pairing ads with different pattern sequences within the same length bucket. Two scopes are supported: *same-cluster* (products from the same semantic cluster, holding domain context constant) and *cross-cluster* (products from different clusters, used to increase task volume toward a target of 1,000 tasks). The current task set contains 977 evaluation tasks.

TEMPLATE RETRIEVAL

Why Not BM25

We first prototyped keyword-based retrieval using BM25. Retrieval quality was poor for two reasons. First, ad copy is very short, typically one to five sentences, making term frequency statistics unreliable. Second, ad copywriters frequently use novel, coined, or domain-specific vocabulary. BM25 fails to match semantically similar templates that use different surface forms for the same concept.

Dense Semantic Retrieval

We index templates in ChromaDB using dense embeddings computed with *text-embedding-3-small*. Each template is stored with its example product description as the document and with category, sequence length, and frequency score as filterable metadata.

Retrieval runs in two stages. Stage 1 applies a compound metadata filter: the category must match the user’s input and the sequence length must fall within the target length bucket. Five length buckets are defined by sequence step count: xs (1–2 steps), s (3), m (4–5), l (6–8), xl (9–15). The top 20 semantic candidates are retrieved from this filtered set. If the category filter returns an empty result, the system falls back to a sequence-length filter with local category matching.

Stage 2 re-ranks the 20 candidates using Bradley-Terry scores when available, or frequency score during the cold-start period. The top 3 templates are returned to the generation API.

BRADLEY-TERRY RANKING

We use the Bradley-Terry model [2] to assign a latent strength score to each template based on accumulated pairwise preference outcomes. The implementation is custom and does not rely on any third-party ranking library.

Algorithm

Given a set of resolved pairwise comparisons, let β_i denote the strength score of template i . The iterative update rule is:

$$\beta_i = \frac{W_i}{\sum_j \frac{n_{ij}}{\beta_i + \beta_j}}$$

where W_i is the total weighted wins of template i , n_{ij} is the total comparison count between i and j , and the sum runs over all opponents. After each iteration, scores are normalized by their mean to keep the scale stable. Convergence is declared when the maximum per-template change falls below 10^{-6} or after 1,000 iterations. Tie outcomes are represented as two half-weight comparisons in opposite directions.

Cold Start and Periodic Re-fitting

When the global comparison graph has fewer than 10 comparisons, the BT fit is skipped and the retriever falls back to frequency score for all templates.

Re-fitting runs in a background task that checks at regular intervals and triggers a full recomputation only when the count of resolved tasks has increased since the last run. Updated scores are written to the database and pushed into the retriever’s in-memory score table, so no restart is required.

GENERATION

Structured Mode

In structured mode the user selects one of the three recommended templates. The system passes the template’s pattern sequence as a hard structural constraint to the language model, together with the product description, category, and any additional user guidance. The system prompt requires the model to generate exactly one sentence

per pattern step and return the result as a JSON array of labeled segments:

```
{ "segments": [
  { "label": "AH", "text": "..."},
  { "label": "PP", "text": "..."},
  ...
]}
```

This representation allows the frontend to display pattern labels alongside each generated sentence, making the structural logic fully visible to the user. This follows the PAIR Guidebook’s principle of explainability [3].

Direct Mode

Direct mode passes the same inputs to the language model with no structural constraint. The target output length is expressed as a word count range derived from the selected length bucket. This mode replicates the experience of using a general-purpose language model for ad writing and serves as a qualitative baseline for side-by-side comparison with the structured output.

Segment-Level Editing

After initial generation, the user can edit the structured output at the segment level. Each segment supports four operations: *regenerate* (rewrite with an optional instruction prompt), *longer* (expand the segment), *shorter* (compress the segment), and *disable* (remove the segment from the output). Segments not marked for editing retain their original text exactly, enforced at the application layer after model response parsing. This supports the PAIR Guidebook’s principle of user control [3], giving users meaningful agency over individual structural moves rather than forcing a full regeneration.

DESIGN PRINCIPLES

The design of AdCraft’s interfaces is based on two ideas from prior work.

The Google PAIR Guidebook [3] identifies transparency and user control as foundational requirements for AI systems that earn user trust. AdCraft applies these principles directly: pattern labels make each structural decision visible and inspectable, and segment-level editing gives users direct control over individual parts of the output.

The evaluation loop follows the interactive machine learning framework from Amershi et al. [1], where users give feedback, the model updates, and users see better results over time. In AdCraft, volunteer evaluators submit pairwise preferences, the Bradley-Terry model updates template scores, and future generation requests retrieve higher-ranked templates. The system makes this feedback cycle visible: contribution counters on the evaluation page show users how their input is affecting the shared ranking.

A secondary design goal is accessibility across user types. The *Show Details* toggle in the navigation bar implements progressive disclosure: when details are enabled, pattern sequence labels, template identifiers, and cluster metadata are displayed throughout both the evaluation and generation views. When details are disabled, only the ad text and action controls are shown. This lets non-technical users, such as marketers or small business owners who are not familiar with rhetorical notation, use the tool without running into unfamiliar

terms, while advanced users can still see all the details when they want. Figure 3 and Figure 4 illustrate this contrast.

The top section, labeled 'PAIR EVALUATION', shows a detailed view with 'Task ID: eval_ff0ac22f2840', 'Task Type: pair', and 'Task Scope: Cross Cluster'. Below this, 'AD SAMPLE A' is shown with 'Ecommerce' category, 'LENGTH: XS - 2', and 'AD ID: 1104f6dc-0c3d-4dc9-b5fb-e006a4de2509'. Further details include 'SC: ecommerce-0003', 'AH', and 'FB'. The bottom section, also labeled 'PAIR EVALUATION', shows a simplified view with 'AD SAMPLE A' and 'Ecommerce' category, but without the additional metadata.

Figure 3: Evaluate page with *Show Details* enabled (top) vs. disabled (bottom). With details on, task ID, task scope, ad ID, cluster ID, and pattern sequence tags are visible. With details off, only category and length tags remain alongside the ad text.

The top section, labeled 'TEMPLATE', shows a detailed view with score metadata: 'SD: 0.780', 'SR: 16', 'L: 3', 'BT: NA', 'FQ: 0.000361141206211629', 'FS: 0.000361141206211629', and 'FR: 12'. Below this, a pattern sequence is shown: 'Social Proof' (blue), 'Call To Action' (blue), and 'Urgency' (red). The bottom section, also labeled 'TEMPLATE', shows a simplified view with the same pattern sequence but without the score metadata.

Figure 4: Generate page template header with *Show Details* enabled (top) vs. disabled (bottom). Score metadata (SD, SR, BT score, frequency score, frequency rank) is visible only in the detailed view; the clean view shows only the color-coded pattern sequence.

EVALUATION

Pairwise Preference Interface

The evaluation page presents two ad samples drawn from the same evaluation task and asks the user to choose which structural approach is more compelling. Users submit one of three responses: A is better, about the same, or B is better. A task is resolved when at least three responses produce a strict majority, or when five responses yield no strict majority (recorded as a tie). Resolved tasks feed into the Bradley-Terry re-fitting cycle.

The interface shows a 'Global Evaluation Progress' header with 'Your Contributions: 61' and 'Global Contributions: 2996'. A progress bar indicates 'Globally resolved: 955 / 977' (98%). Below this, the 'PAIR EVALUATION' section shows two ad samples side-by-side. 'AD SAMPLE A' is 'Ecommerce' with 'LENGTH: XS - 2' and text: 'Whatnot: Shop, Sell, Connect. Explore a wide selection of streetwear brands with round-the-clock deals. Live on Whatnot'. 'AD SAMPLE B' is 'Ecommerce' with 'LENGTH: XS - 2' and text: '([product.current_price.strip_zeros]) - ([product.name]) Up to 70% Off Storewide - End of Season Sale Starts Now! Limited Time Only!'. At the bottom, three buttons are shown: 'A is better', 'About the same', and 'B is better'.

Figure 5: Pairwise evaluation interface. The progress bar shows global task completion. Two ad samples appear side by side; three buttons below collect the vote. Per-session and global contribution counters are shown in the header.

Current Scale

Metric	Value
Structural templates	1,450
Evaluation tasks	977
Volunteer responses	1,862
Resolved tasks	951
Template BT coverage	89%

Templates with fewer than 10 accumulated comparisons do not yet have a stable Bradley-Terry score and continue to be ranked by frequency score. These represent the long-tail of sequences that appeared only once or twice in the original corpus.

User Study

We recruited volunteer participants, primarily UIUC students, in two roles. Fifteen volunteers participated as pairwise evaluators, submitting votes through the evaluation interface. Twenty-five volunteers explored the generation interface to try structured ad copy creation. Participation was informal and exploratory; no formal study protocol or questionnaire was administered, so all collected feedback is qualitative.

The 15 evaluators collectively submitted 1,862 pairwise responses, resolving 951 of the 977 evaluation tasks (97% completion).

Participants who used the generation interface reported consistent findings across three dimensions. First, structured mode gave them a stronger sense of participation and control compared to direct generation; seeing the pattern label for each sentence and being able to edit individual sections changed the interaction from passively receiving output to actively shaping it. Second, working with visible pattern labels helped participants understand the rhetorical logic behind effective ad copy, providing learning value that unconstrained generation cannot offer. Third, participants found the generation interface intuitive and easy to use, with segment-level editing highlighted as the most useful feature.

HOW TO USE ADCRAFT

AdCraft is accessible at <https://adcraft.jnxy.net>. No account or login is required.

Evaluating Ad Pairs

The default view on load is the Evaluate page (Figure 5). Users are presented with two ad samples, A and B, and select which structural approach they find more compelling. Clicking any of the three response buttons (A is better, About the same, B is better) submits the response and immediately loads the next unanswered task. Users can click *Switch Pair* at any time to skip the current task and load a different one.

Two counters appear in the header: *Your Contributions* tracks votes submitted by the current session, and *Global Contributions* shows the total vote count across all sessions. A progress bar beneath the header displays the fraction of tasks globally resolved. The *Show Details* toggle in the navigation bar reveals additional metadata (task ID, task scope, ad ID, cluster identifier, and the full pattern sequence for each sample) for users who want to inspect the technical structure. Disabling it presents a clean view with only category and length

tags alongside the ad text, suitable for users without a technical background.

Generating Structured Ad Copy

The generation page (Figure 6) has three panels: a session list on the left, query configuration and template selection in the center, and the generation result on the right.

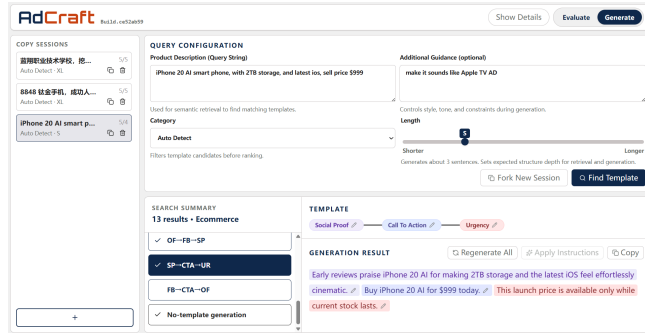


Figure 6: Full generation interface. Left: copy session list. Center: query configuration and ranked template list. Right: generation result with color-coded pattern segments and inline edit controls.

The workflow proceeds as follows:

1. Enter a product description in the *Product Description* field (Figure 7). Optionally, add a tone or style directive in the *Additional Guidance* field (for example, “make it sound like an Apple TV ad”).
2. Select a product category from the *Category* dropdown, or leave it as *Auto Detect* to let the system infer the category from the description.
3. Drag the *Length* slider to set the target output length. The slider steps through five presets from shorter to longer outputs.
4. Click *Find Template* to retrieve top ranked structural templates.

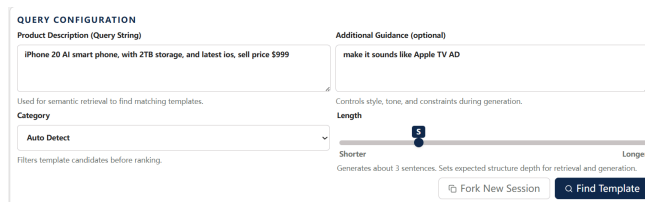


Figure 7: Query configuration panel. *Product Description* drives semantic retrieval; *Additional Guidance* controls style and tone during generation. The *Length* slider steps through five length presets.

1. Select a template from the results list. Each entry shows its rhetorical pattern sequence. A *No-template generation* option is also available for unconstrained direct-mode output. Once selected, the system generates structured copy on the right panel.
2. Review the structured output (Figure 8). Each sentence is displayed with its color-coded pattern label. Click the edit icon on any segment to choose: *Regenerate* (with an optional custom prompt), *Make it Longer*, *Make it Shorter*, or *Disable this section*.
3. Click *Apply Instructions* to update only the modified segments. Unchanged segments are preserved exactly. Click *Regenerate All* to rewrite the full output from scratch.

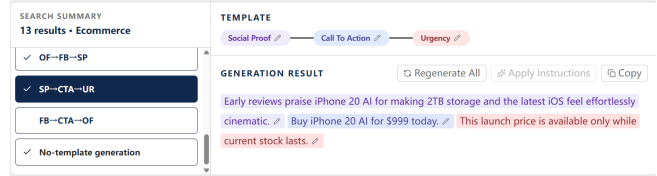


Figure 8: Template selection and generation result. The selected template’s pattern chain is shown as color-coded pills. Generated segments are color-matched to their pattern labels, each with an inline edit icon.

The left sidebar lists all copy sessions in the current browser session. Each session stores its query, template selection, and generated results independently. The *Fork New Session* button creates a new session pre-populated with the current query, useful for exploring variations. The *Show Details* toggle can be turned off to hide pattern labels and score metadata, presenting a clean interface for users without an advertising background.

TEAM CONTRIBUTIONS

All four team members participated in pairwise ad evaluation as annotators throughout the project, contributed to UI brainstorming and design discussions, and helped test the deployed prototype.

Kaiyang Teng (project coordinator) led the overall system architecture design, provisioned and managed the production server, oversaw data collection from the Meta Content Library, and implemented the offline processing pipeline including deduplication and semantic clustering. Kaiyang also contributed to backend API design and coordinated cross-member integration.

Hengrui Ren designed the rhetorical pattern taxonomy, implemented the LLM annotation pipeline with concurrent processing and checkpoint resume, and constructed the structural template library. Hengrui coordinated the recruitment and onboarding of volunteer evaluators and assisted with evaluation task quality review.

Ziqin Shen implemented the template retrieval system, evaluated the initial BM25 prototype, designed the two-stage dense retrieval pipeline with ChromaDB, and implemented the custom Bradley-Terry ranking algorithm. Ziqin also assisted with backend API integration and deployment verification.

Yipeng Xin built the frontend single-page application, implemented the backend REST API, designed the pairwise evaluation and segment-level generation interfaces, and managed the Docker-based deployment pipeline. Yipeng also contributed to the system architecture and assisted with pipeline integration testing.

CONCLUSION

AdCraft shows that making the rhetorical structure of AI-generated ad copy visible and editable is practical and useful. Users get real control over the writing strategy, not just the final output. The evaluation loop, based on the interactive machine learning framework [1], connects user preferences to template rankings in a transparent way.

The deployed prototype is available at <https://adcraft.jnxy.net> and the source code and documentation are at <https://github.com/jnxy/AdCraft>.

The primary direction for future work is a controlled quantitative evaluation of structured versus unstructured generation quality.

The planned approach uses product descriptions already inferred during pipeline annotation to produce matched output triples: one structured LLM output, one direct LLM output, and the original real ad from which the description was derived. Pairwise comparison tasks are constructed from all three pairings within each triple and deployed through the existing evaluation interface for volunteer rating. Including the original human-authored ad as a control condition is essential: it allows the analysis to separate the effect of rhetorical structure from any general preference lift or penalty that may arise simply because the text was generated by a language model. Statistical analysis of the resolved pairwise outcomes will then quantify whether structured generation consistently outperforms unstructured generation, and to what degree, relative to the human-authored baseline.

REFERENCES

- [1] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. 2014. Power to the People: The Role of Humans in Interactive Machine Learning. *AI Magazine* 35, 4 (2014), 105–120.
- [2] Ralph Allan Bradley and Milton E. Terry. 1952. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika* 39, 3/4 (1952), 324–345.
- [3] Google PAIR. 2019. People + AI Guidebook.
- [4] Henry A. Laskey, Ellen Day, and Melvin R. Crask. 1989. Typology of Main Message Strategies for Television Commercials. *Journal of Advertising* 18, 1 (1989), 36–41.
- [5] Meta Platforms, Inc. 2024. Meta Content Library and API.